HONEYWELL SOFTWARE ENGINEERING TEAMS — IA · BA · PA&T

AI Champions Programme — GitHub Copilot Enterprise, SDD, MCP & Agentic Software Engineering

Module 06

Test Strategy & Validation

AI can write code and tests faster than you can read them. This module builds the testing stack that proves the code matches the spec — from unit tests and test doubles to integration, contract and regression checks.

DURATION
1.5 hours

FORMAT
Concepts + defect-injection lab

PRIMARY TOOL
Copilot + your test framework

How We'll Spend These 90 Minutes

Build the stack from the bottom up, then prove it works by breaking something on purpose.



01

The Testing Stack & Spec Traceability

15 min

Why AI-era tests must come from the spec; the layers; mapping acceptance criteria to tests.



02

Unit Tests & Test Doubles

15 min

Spec-derived unit tests with Copilot; dummies, stubs, fakes, mocks and spies.



03

Integration, API & Database Validation

15 min

Real wiring with lightweight boundaries; checking responses and stored state together.



04

Negative, Contract & Regression

15 min

Break it on purpose; guard the contract; pin behaviour that must not change.



05

Suite Quality & Best Practices

10 min

Coverage vs acceptance-criteria coverage vs mutation; do/don't for AI-assisted testing.



06

Lab: Inject a Defect, Prove It's Caught

20 min

Derive a strategy, write tests with doubles, plant a bug, and show the suite catches it.

Module 06 in the 12-Module Journey

Specs from Modules 04–05 become tests here, and tests become the gate in PR automation next.

FOUNDATION			BUILD & VALIDATE			EXTEND WITH TOOLS & AGENTS				GOVERN	CAPSTONE
1	2	3	4	5	**6**	7	8	9	10	11	12
Objectives & GenAI Recap	Copilot Advanced	Prompt & Context Eng.	Spec-Driven Dev (Spec Kit)	SDD to Full SDLC	**Test Strategy & Validation**	MCP & Custom MCP Server	PR Automation & LLM-Judge	No-Code Agents (Rovo)	Code Agents & Simple RAG	Observability, ROI & Gov.	Capstone & Roadmap

M04–05
Acceptance criteria in the spec are the source of every test

M05
Characterization and regression tests protect brownfield change

M08
PR automation checks test evidence against the spec

M11
Defect-catch and rework data feed observability and ROI

Why this matters: the faster AI writes code, the more your safety depends on tests you trust. This module is how you earn that trust.

Why It Matters: Tests Must Come from the Spec, Not the Code

If AI writes both the code and the tests from the same misunderstanding, everything passes and the bug ships.

TESTS DERIVED FROM CODE — A FALSE SENSE OF SAFETY

1 · Code generated

Model fills a gap in the requirement with a guess

→

2 · Tests from the code

Tests assert whatever the code already does

→

3 · All green

Tests and code share the same wrong assumption

→

4 · Bug ships

Nobody checked it against the intent

TESTS DERIVED FROM THE SPEC — AN INDEPENDENT CHECK

1 · Acceptance criteria

Written and reviewed before any code

→

2 · Tests written first

Run them: they should fail (red)

→

3 · Code makes them pass

Green means the criteria are met

→

4 · Bug caught

A wrong guess fails a test written from the intent

Rule of thumb: a test that has never failed has never proved anything. See it fail first — then make it pass.

The Testing Stack: Many Fast Tests, Few Slow Ones

Each layer checks something the layer below cannot. Build from the bottom up.



	WHAT IT CHECKS	SPEED · COUNT
Acceptance / end-to-end	Whole feature through the real entry point	Slowest · fewest
API & contract	HTTP surface and schemas other systems rely on	Slow
Integration	Real wiring: service + repository + database	Medium
Unit	One behaviour, dependencies replaced by doubles	Fastest · most

Regression is not a layer: it is the same layers re-run on every change. A regression suite is simply the tests you refuse to delete — Module 05's pinned behaviour.

Spec-to-Test Traceability: Every Criterion Has a Test

Name tests after acceptance criteria so gaps are visible — to you, to reviewers and to CI.

Acceptance criteria

AC-1…AC-n with IDs, in spec.md

Test case list

Happy, negative and boundary cases per AC

Test code

Named with the AC ID; written first

Evidence

CI run + matrix in the PR

traceability.md (illustrative)

```
AC-1  refund <= original amount
  UT-1  Refund_Over_Total_Fails            (unit)
  UT-2  Refund_Exactly_Total_Succeeds      (boundary)
AC-2  only Paid orders can be refunded
  UT-3  Refund_Unpaid_Order_Fails          (negative)
AC-3  audit entry written
  IT-1  Refund_Writes_Audit_Row            (integration)
AC-4  error code REFUND_EXCEEDS_TOTAL  ← NO TEST YET
```

A gap you can see

AC-4 has no test. In a matrix that is one visible line; in a pile of AI-generated tests it is invisible. Ask Copilot to list ACs with no matching test — then review its answer.

Best practice: keep the matrix in the repo and in the PR. A reviewer should be able to answer “is every criterion tested?” in under a minute.

Deriving the Test Strategy from the Feature

For each criterion, choose the cheapest layer that can prove it. Illustrative example: the refund feature.

Acceptance criterion	Unit	Integration	API	Contract
AC-1 Refund ≤ original amount	●		○	
AC-2 Only Paid orders can be refunded	●	○	●	
AC-3 Audit entry is written	○	●		
AC-4 Error code REFUND_EXCEEDS_TOTAL	●		●	●
AC-5 Billing API v1 responses unchanged			○	●

● *primary check* ○ *also covered*

Unit Doubles replace repo and payment gateway

Integration Real database engine; external API stubbed

API / Contract Real HTTP surface; schema checked against the contract

Best practice: one criterion, one primary layer. If every criterion is tested at every layer you have a slow suite; if one is tested nowhere you have a hole.

Unit Tests with Copilot: Prompt from the Spec

Ask for test cases first, from the criteria — before the model reads the implementation.

THE PROMPT PATTERN

chat prompt (illustrative)

```
Using #file:spec.md (AC-1 to AC-4) only:
1. List test cases: happy, negative,
   boundary — name each with its AC ID.
2. Write xUnit tests. Use a stub for the
   repository and payment gateway.
3. Do NOT open RefundService.cs yet.
4. Run the tests; they must FAIL first.
```

THE RESULT — ARRANGE · ACT · ASSERT

RefundTests.cs (illustrative)

```
[Fact]  // AC-1
void Refund_Over_Total_Fails()
{
  var order = PaidOrder(total: 100m);    // arrange
  var r = svc.Refund(order, 150m);       // act
  Assert.Equal("REFUND_EXCEEDS_TOTAL",
               r.Error.Code);            // assert
}
```

One behaviour per test

A failure points at one cause

Name by criterion

AC ID in the name or a comment

Review like code

Tests that assert nothing still pass

Best practice: withhold the implementation until the tests exist. Then show Copilot the code and ask it to find criteria the tests miss.

Test Doubles: From Dummy to Mock

Stand-ins for real dependencies, from the simplest to the most assertive. Use the simplest one that works.

STATE-BASED — check results →

BEHAVIOUR-BASED — check interactions

1

Dummy

Fills a parameter; never actually used

```
new NullLogger()
```

2

Stub

Returns canned answers so the code under test can proceed

```
gateway → Approved
```

3

Fake

A real but lightweight implementation

```
in-memory repository
```

4

Spy

A stub that also records how it was called

```
captured: Charge(150)
```

5

Mock

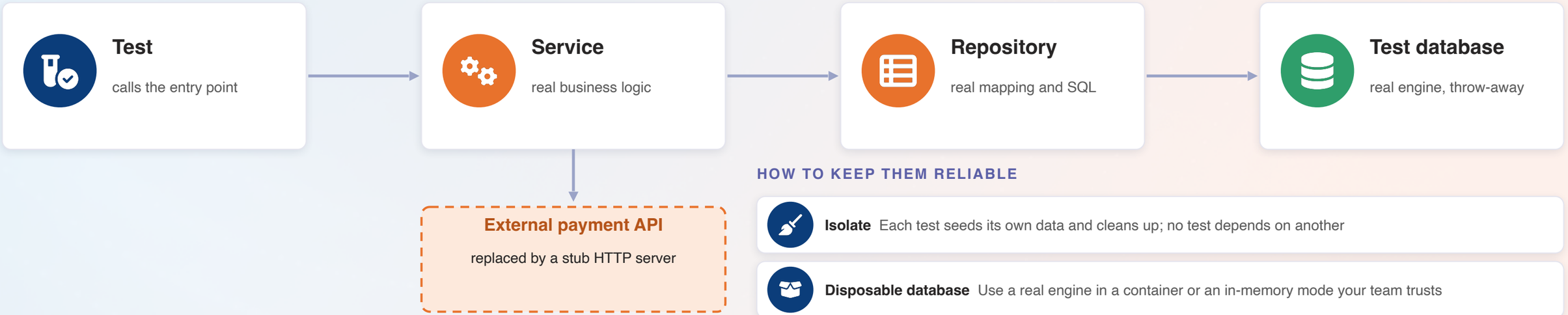
Pre-set expectations; fails if they are not met

```
expect audit.Write once
```

Best practice: double only at the boundary — network, clock, payments, email. Mock everything and your tests describe the implementation, so every refactor breaks them.

Integration Tests: Real Wiring, Controlled Edges

Keep the parts you own real. Replace only what you cannot control or what is slow.



HOW TO KEEP THEM RELIABLE

- **Isolate** Each test seeds its own data and cleans up; no test depends on another
- **Disposable database** Use a real engine in a container or an in-memory mode your team trusts
- **Test failure paths** Make the stub time out or return 500 — see what the service does

Best practice: integration tests prove the wiring that unit tests with doubles cannot: mappings, queries, transactions, configuration.

API and Database Validation: Check the Response and the Stored State

A good API test asserts what came back and what did — or did not — change underneath.

ONE TEST, TWO WITNESSES

RefundApiTests (illustrative, pseudo-code)

```
POST /orders/42/refund   { "amount": 150 }

expect status       == 422
expect body.code    == "REFUND_EXCEEDS_TOTAL"
expect body matches ErrorSchema

expect db.orders[42].refunded == 0   // unchanged
expect db.audit rows          == 0   // no side effect
```

API checks

- Status codes for success and each failure
- Response matches the schema
- Error shape is stable and documented
- Auth: allowed and forbidden callers

Database checks

- State after success is exactly right
- Failure leaves state untouched (rollback)
- Constraints and uniqueness hold
- Migrations apply to a clean database

Best practice: for every failing request, also assert nothing changed. Half-applied writes are the bugs that cost the most to find later.

Illustrative pseudo-code; use your team's API test tool (HTTP client, collection runner, or framework test host).

Break It on Purpose: Negative and Boundary Scenarios

AI generates happy paths readily. You have to ask for the rest — one prompt per row.

Invalid input

Null, empty, wrong type, too long, malformed

Boundaries

Exactly at, one below, one above every limit

Not allowed

Unauthenticated, wrong role, someone else's data

Dependency fails

Timeout, 500, slow response, empty reply

Repeat and overlap

Duplicate request, retry, two at once

Wrong state

Refund an unpaid or already-refunded order

prompt (illustrative)

```
For AC-2, list negative and boundary cases I haven't covered.
```

Each failure scenario must assert a clear error and no side effects — not just “doesn't crash”.

Best practice: the six rows are a reusable checklist. Paste it into a prompt file and run it against every new acceptance criterion.

Contract Validation: Protect Everyone Who Depends on You

A contract is the promise your API makes. Test it from both sides so a change can't break a consumer silently.

The contract

API description or schema, derived from the spec's interfaces section

Provider test

Does the real API still match the contract?

Consumer test

Does the client work against a stub built from the contract?

In CI, on every change

A breaking change fails the build before it reaches a consumer — not after.

SAFE CHANGES

- Add an optional field
- Add a new endpoint
- Loosen a validation rule

BREAKING CHANGES

- Remove or rename a field
- Change a field's type
- Add a new required input

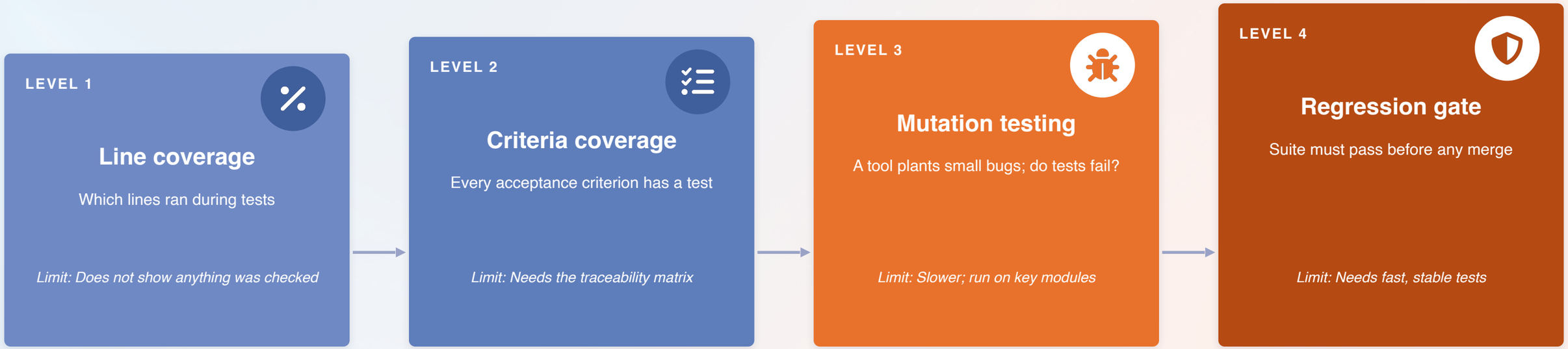
Best practice: version the contract with the spec. A contract change is a spec change — it needs a review before any code.

Is the Suite Any Good? Four Levels of Evidence

Passing tests prove little unless the tests themselves are good. Climb the ladder.

BASIC — what ran

ADVANCED — does the suite catch bugs?



Today's lab is mutation testing by hand: you plant one defect and see whether the suite notices. Automated mutation tools do the same at scale. Agree your own coverage thresholds as a team — there is no universal number.

Best Practices for AI-Assisted Testing

Treat AI as a fast test writer and a poor judge of what the tests should prove.

DO THIS	AVOID THIS
Derive tests from the spec's acceptance criteria, with IDs	Ask Copilot to “write tests for this file” and stop there
See each new test fail before the code makes it pass	Accept tests that assert nothing or only that code ran
Ask explicitly for negative, boundary and failure cases	Weaken or delete a failing test to get a green build
Double only at boundaries; prefer fakes and state checks	Mock every collaborator until tests mirror the code
Assert on state and side effects, not just return values	Chase a coverage percentage as the goal
Review generated tests as carefully as generated code	Let AI edit tests and implementation in one unreviewed step

Golden rule: when a test fails, decide whether the code or the spec is wrong — never silently change the test to match whatever the code does.

Lab: Inject a Defect, Prove the Suite Catches It

LAB 1 · 20 min

Derive the strategy, write the tests with doubles, then break the code on purpose.

Goal Build a spec-derived test suite for your SDD feature, plant one deliberate defect, and show the suite fails for the right reason.

STEPS

1. From spec.md, build the traceability matrix (AC → layer) with Copilot, then adjust it by hand. Mark any criterion with no test.
2. Write failing unit tests first (stubs/fakes at the boundary), then integration or API tests for the criteria that need them.
3. Make everything green. Record the baseline: tests, pass rate, AC coverage.
4. Plant one defect (e.g. relax a limit, drop a check). Run the suite, find which test fails, then fix the code and re-run.

FILES YOU CREATE

repo/

```
lab06/
  traceability.md
  tests/unit/
  tests/integration/
  defect-report.md
```

DONE WHEN

defect-report.md names the planted defect, the test that caught it, and any criterion that would have slipped through.

What You Can Do Now

Six outcomes from today — and the habit that ties them together: tests come from the spec, and must fail before they pass.

- ✓ Acceptance criteria are mapped to tests through a traceability matrix
- ✓ Spec-derived unit tests are written with the right test doubles
- ✓ Integration and API tests validate wiring, responses and stored state
- ✓ Negative, boundary and failure scenarios are requested and covered
- ✓ Contract and regression checks protect consumers and existing behaviour
- ✓ A planted defect proved the suite catches real bugs

NEXT: MODULE 07 — MCP-ENABLED WORKFLOWS & CUSTOM MCP SERVER CREATION · 3 HOURS

Connect agents to repositories, APIs and enterprise services through MCP, then build and govern a simple custom MCP server.

THE TESTING STACK TODAY

Unit → Integration → API → Contract → Regression

Questions & Discussion

Module 06 — Test Strategy & Validation